



OUTLINE / TÓM TẮT DỰ ÁN

XÂY DỰNG WEBSITE PHÂN LOẠI VÀ PHÁT HIỆN BỆNH CỦA CÂY TRỒNG THÔNG QUA LÁ CÂY SỬ DỤNG TRÍ TUỆ NHÂN TẠO

Sinh viên: — MSV:

Ngành: — Khóa:

Giảng viên hướng dẫn:

CHƯƠNG 1: MỞ ĐẦU

1.1. Lý do chọn đề tài

- Nông nghiệp là ngành kinh tế trọng điểm của Việt Nam. Bệnh cây trồng gây thiệt hại lớn về năng suất và chất lượng nông sản.
- Việc phát hiện bệnh truyền thống phụ thuộc vào kinh nghiệm của nông dân, dễ sai sót và chậm trễ.
- Trí tuệ nhân tạo (AI), đặc biệt là **Thị giác máy tính (Computer Vision)** và **Mạng nơ-ron tích chập (CNN)**, đã chứng minh khả năng nhận diện bệnh cây trồng với độ chính xác cao qua hình ảnh.
- Nhu cầu xây dựng **ứng dụng web** giúp nông dân và kỹ sư nông nghiệp dễ dàng tiếp cận công nghệ AI mà không cần kiến thức chuyên sâu.

1.2. Mục tiêu đề tài

- Mục tiêu chính:** Xây dựng hệ thống website ứng dụng trí tuệ nhân tạo để phân loại và phát hiện bệnh cây trồng qua ảnh lá cây.
- Mục tiêu cụ thể:**
 - Huấn luyện mô hình CNN (MobileNetV2) phân loại **15 loại bệnh** trên 3 loại cây trồng (cà chua, ớt chuông, khoai tây) đạt **độ chính xác ~99%**
 - Xây dựng **REST API** backend bằng FastAPI cho phép inference model AI qua HTTP

- iii. Xây dựng **giao diện web** frontend bằng NextJS 16 với tính năng upload ảnh, chụp ảnh trực tiếp từ camera, và hiển thị kết quả phân tích
- iv. Xây dựng **Dashboard** thống kê, lịch sử phân tích, biểu đồ phân bố bệnh
- v. Xuất model sang **ONNX** để triển khai nhẹ (chạy trên CPU, không cần GPU)

1.3. Phạm vi đề tài

- **Đối tượng:** 3 loại cây trồng: Cà chua, Ớt chuông, Khoai tây
- **Số lớp phân loại:** 15 lớp (12 bệnh + 3 khỏe mạnh)
- **Dữ liệu:** Bộ dữ liệu PlantVillage (công khai trên Kaggle)
- **Nền tảng:** Ứng dụng Web (responsive, hoạt động trên cả desktop và mobile)

1.4. Phương pháp nghiên cứu

- Nghiên cứu lý thuyết về CNN, Transfer Learning, MobileNetV2
- Thu thập và tiền xử lý dữ liệu từ PlantVillage dataset
- Huấn luyện và đánh giá model trên Google Colab (GPU T4/P100)
- Thiết kế và phát triển hệ thống web full-stack
- Kiểm thử và đánh giá hiệu năng hệ thống

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1. Tổng quan về Trí tuệ nhân tạo và Học sâu (Deep Learning)

- Khái niệm AI, Machine Learning, Deep Learning
- Mối quan hệ và sự phát triển của các lĩnh vực

2.2. Mạng nơ-ron tích chập (Convolutional Neural Network — CNN)

- Kiến trúc CNN: Convolutional Layer, Pooling Layer, Fully Connected Layer
- Các khái niệm: Feature Map, Stride, Padding, Activation Function (ReLU)
- Batch Normalization, Dropout

2.3. Transfer Learning — Học chuyển giao

- Khái niệm và lợi ích của Transfer Learning

- Pre-trained model trên ImageNet
- Fine-tuning vs Feature Extraction

2.4. MobileNetV2

- Kiến trúc MobileNetV2: **Inverted Residual Block**, **Depthwise Separable Convolution**
- Ưu điểm: nhẹ, nhanh, phù hợp triển khai trên thiết bị có tài nguyên hạn chế
- So sánh với các kiến trúc khác (ResNet, EfficientNet, VGG)

2.5. ONNX (Open Neural Network Exchange)

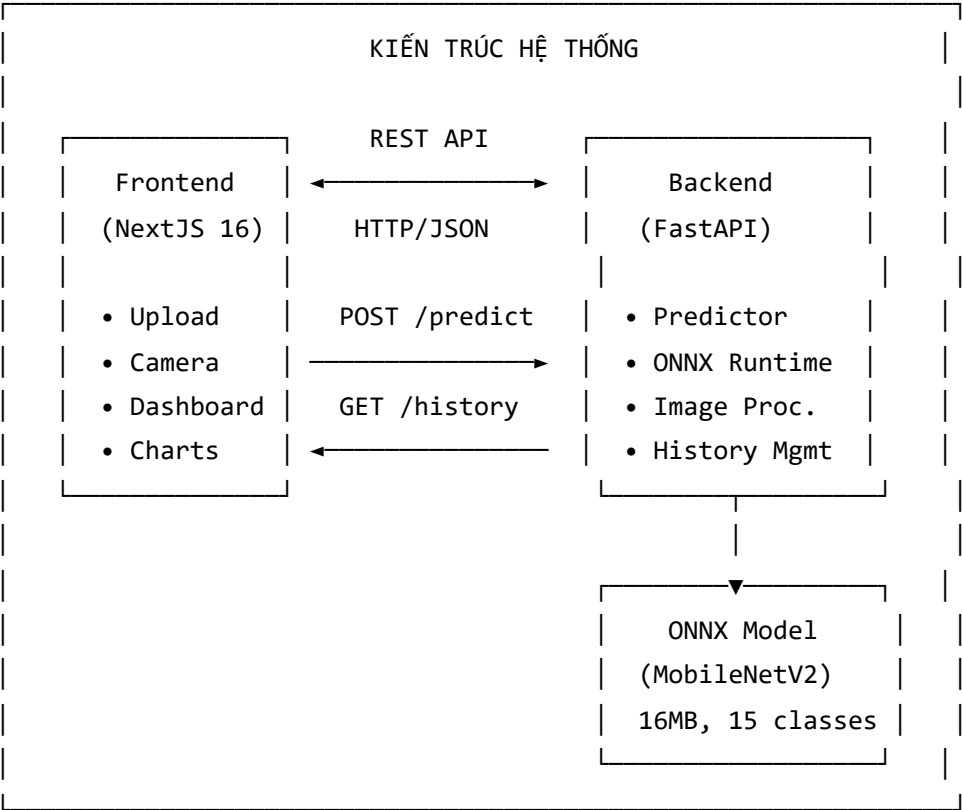
- Khái niệm ONNX: chuẩn mở để biểu diễn model AI
- Ưu điểm: cross-platform, tối ưu inference, không phụ thuộc framework
- ONNX Runtime: engine chạy inference hiệu quả trên CPU

2.6. Bệnh cây trồng

- Tổng quan về các loại bệnh trên cà chua, ớt chuông, khoai tây
- Phương pháp phát hiện bệnh truyền thống vs AI

CHƯƠNG 3: PHÂN TÍCH VÀ THIẾT KẾ HỆ THỐNG

3.1. Kiến trúc tổng thể hệ thống



3.2. Thiết kế Backend (FastAPI)

3.2.1. Công nghệ sử dụng

Thư viện	Phiên bản	Vai trò
FastAPI	0.115.0	Web framework REST API
Uvicorn	0.30.0	ASGI Server
ONNX Runtime	1.19.0	Inference model AI
Pillow	10.4.0	Xử lý ảnh (resize, convert)
NumPy	1.26.4	Tiền xử lý tensor, softmax
python-multipart	0.0.9	Xử lý file upload multipart

3.2.2. API Endpoints

Method	Endpoint	Mô tả	Input	Output
GET	/api/health	Kiểm tra trạng thái server	—	Status, num_classes, timestamp
GET	/api/classes	Danh sách 15 lớp bệnh	—	Tên VN/EN, mô tả, khuyến nghị
POST	/api/predict	Phân loại bệnh từ ảnh	Ảnh (multipart)	Kết quả dự đoán + Top 5
GET	/api/history	Lịch sử phân tích	—	History list + thống kê

3.2.3. Module Predictor (predictor.py)

- **Load model ONNX** khi khởi động server (1 lần duy nhất)
- **Tiền xử lý ảnh:** RGB convert → Resize (256×256) → Normalize (ImageNet mean/std) → HWC→CHW→NCHW
- **Inference:** Chạy ONNX session → Softmax → Top 5 predictions
- **Thông tin bệnh:** Dictionary 15 mục chứa tên tiếng Việt, mô tả triệu chứng, khuyến nghị xử lý

3.2.4. Quản lý lịch sử

- Lưu trữ **in-memory** (tối đa 50 bản ghi gần nhất)
- Lưu ảnh upload vào thư mục uploads/ và serve qua static files
- Tính toán thống kê: tổng mẫu, số lá khỏe, số lá bệnh, phân bố theo loại bệnh

3.3. Thiết kế Frontend (NextJS 16)

3.3.1. Công nghệ sử dụng

Công nghệ	Phiên bản	Vai trò
Next.js	16.1.6	React framework (App Router)
React	19.2.3	UI library
TypeScript	5.x	Type-safe JavaScript
Vanilla CSS	—	Styling (dark mode, glassmorphism)

3.3.2. Cấu trúc trang

Trang	Route	Tính năng
Trang Phân tích	/	Upload ảnh kéo thả, chụp ảnh trực tiếp, hiển thị kết quả AI
Dashboard	/dashboard	Stat cards, biểu đồ phân bố bệnh (bar chart), tỷ lệ bệnh (donut chart), bảng lịch sử

3.3.3. Tính năng chính

Trang Phân tích (/):

- **Upload ảnh:** Kéo thả (Drag & Drop) hoặc click chọn file — hỗ trợ JPG, PNG, WEBP
- **Camera trực tiếp:** Mở webcam/camera điện thoại, chụp ảnh lá cây gửi phân tích — hỗ trợ chuyển camera trước/sau
- **Hiển thị kết quả:**
 - Trạng thái: Khỏe mạnh  / Phát hiện bệnh 
 - Tên bệnh (tiếng Việt)
 - Thanh độ tin cậy (confidence bar) với animation
 - Mô tả triệu chứng bệnh
 - Khuyến nghị xử lý
 - Bảng **Top 5 dự đoán** với confidence %

Dashboard (/dashboard):

- **4 Stat Cards:** Tổng ảnh phân tích, Lá khỏe mạnh, Lá có bệnh, Số loại bệnh phát hiện
- **Biểu đồ Bar Chart:** Phân bố bệnh theo loại (custom SVG)
- **Biểu đồ Donut Chart:** Tỷ lệ bệnh vs khỏe (custom SVG)
- **Bảng lịch sử:** Ảnh thumbnail, kết quả, trạng thái, độ tin cậy, thời gian
- **Auto-refresh:** Tự động cập nhật mỗi 5 giây

3.3.4. Thiết kế giao diện (UI/UX)

- **Design System:** Dark mode làm chủ đạo, sử dụng CSS Variables cho theming
- **Glassmorphism:** backdrop-filter blur, border semi-transparent
- **Color Palette:** Gradient xanh lá - vàng chanh - vàng (liên tưởng nông nghiệp)
- **Typography:** Font Inter (Google Fonts), 8 font-weight levels
- **Animations:** fadeInUp, pulse loading, smooth transitions (cubic-bezier)
- **Responsive:** Hỗ trợ mobile (≤768px), tablet, desktop via media queries

CHƯƠNG 4: MÔ HÌNH AI VÀ QUÁ TRÌNH HUẤN LUYỆN

4.1. Bộ dữ liệu (Dataset)

- **Nguồn:** PlantVillage Dataset (Kaggle)

- **Số lớp:** 15 classes

- **Các lớp bệnh:**

| # | Class name | Tên tiếng Việt | Cây |

|---|-----|-----|-----|

| 1 | Pepper__bell__Bacterial_spot | Ớt chuông — Đốm vi khuẩn | Ớt chuông |

| 2 | Pepper__bell__healthy | Ớt chuông — Khỏe mạnh | Ớt chuông |

| 3 | Potato__Early_blight | Khoai tây — Cháy lá sớm | Khoai tây |

| 4 | Potato__Late_blight | Khoai tây — Cháy lá muộn | Khoai tây |

| 5 | Potato__healthy | Khoai tây — Khỏe mạnh | Khoai tây |

| 6 | Tomato_Bacterial_spot | Cà chua — Đốm vi khuẩn | Cà chua |

| 7 | Tomato_Early_blight | Cà chua — Cháy lá sớm | Cà chua |

| 8 | Tomato_Late_blight | Cà chua — Cháy lá muộn | Cà chua |

| 9 | Tomato_Leaf_Mold | Cà chua — Mốc lá | Cà chua |

|10 | Tomato_Septoria_leaf_spot | Cà chua — Đốm lá Septoria | Cà chua |

|11 | Tomato_Spider_mites_Two_spotted_spider_mite | Cà chua — Nhện đỏ hai chấm | Cà chua |

|12 | Tomato__Target_Spot | Cà chua — Đốm mục tiêu | Cà chua |

|13 | Tomato__Tomato_YellowLeaf__Curl_Virus | Cà chua — Virus xoắn vàng lá | Cà chua |

|14 | Tomato__Tomato_mosaic_virus | Cà chua — Virus khảm | Cà chua |

|15 | Tomato_healthy | Cà chua — Khỏe mạnh | Cà chua |

- **Chia dữ liệu:** Train / Validation / Test (thường 70/15/15 hoặc 80/10/10)

4.2. Tiền xử lý dữ liệu (Data Preprocessing)

- **Resize:** 256×256 pixels
- **Normalize:** ImageNet statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225])
- **Data Augmentation:** Random flip, rotation, color jitter (trong quá trình training)

4.3. Kiến trúc mô hình

- **Backbone:** MobileNetV2 (pre-trained trên ImageNet)

- **Transfer Learning:** Fine-tuning toàn bộ model
- **Classifier head:** Thay thế FC cuối cùng → Linear(1280, 15)
- **Input shape:** (batch, 3, 256, 256)
- **Output:** 15 logits → Softmax → Probability distribution

4.4. Quá trình huấn luyện

- **Platform:** Google Colab (GPU NVIDIA T4/P100)
- **Framework:** PyTorch
- **Optimizer:** AdamW (lr=1e-4)
- **Loss function:** CrossEntropyLoss
- **Epochs:** 30
- **Batch size:** 32
- **Training notebooks:**
 - i. plant-disease-classification-efficientnet-resnet50.ipynb — Thử nghiệm EfficientNet + ResNet50
 - ii. plant-disease-classifier-pytorch-custom-cnn.ipynb — Custom CNN với PyTorch

4.5. Xuất model sang ONNX

- Chuyển đổi PyTorch model (best_model.pth , ~16MB) → ONNX format (plant_disease_model.onnx , ~16MB)
- Sử dụng torch.onnx.export() với dynamic axes cho batch dimension
- Kiểm tra tính nhất quán (consistency check) giữa PyTorch output và ONNX output

4.6. Các model artifacts đã tạo

File	Kích thước	Mô tả
best_model.pth	16.2 MB	PyTorch weights (training output)
efficientnet_model.pth	16.4 MB	EfficientNet weights (thử nghiệm)
plant_disease_model.onnx	16.2 MB	ONNX model (dùng cho deployment)
class_names.json	407 B	Danh sách 15 class names
model_config.json	205 B	Cấu hình model (image_size, num_classes)
label_encoder.pkl	2.8 KB	Label encoder (sklearn)
inference_transform.pkl	930 B	Transform pipeline cho inference

File	Kích thước	Mô tả
learning_curves.png	57 KB	Biểu đồ loss/accuracy qua các epoch

CHƯƠNG 5: KẾT QUẢ VÀ ĐÁNH GIÁ

5.1. Kết quả huấn luyện model

- **Accuracy trên tập validation:** ~99%
- **Biểu đồ Learning Curves:** (tham chiếu `models/learning_curves.png`)
 - Train Loss vs Validation Loss
 - Train Accuracy vs Validation Accuracy
- **Confusion Matrix:** (nếu có, trong `models/__results__files/`)
- **Classification Report:** Precision, Recall, F1-score cho từng lớp

5.2. So sánh các kiến trúc model

Kiến trúc	Accuracy	Kích thước model	Thời gian inference
MobileNetV2	~99%	16.2 MB	Nhanh (CPU friendly)
EfficientNet	~99%	16.4 MB	Trung bình
ResNet50	(kết quả)	(kích thước)	Chậm hơn
Custom CNN	(kết quả)	(kích thước)	(kết quả)

5.3. Demo hệ thống web

- **Trang Phân tích:** Upload ảnh lá → Hiển thị kết quả dự đoán + confidence + khuyến nghị
- **Trang Dashboard:** Biểu đồ thống kê, bảng lịch sử phân tích
- *(Chụp screenshot các trang hoặc quay video demo)*

5.4. Đánh giá hiệu năng API

Metric	Giá trị
Thời gian load model	1 lần khi khởi động server

Metric	Giá trị
Thời gian inference	~100-300ms / ảnh (CPU)
API response time	~500ms - 1s (bao gồm upload + inference)
Kích thước model ONNX	16.2 MB
Bộ nhớ sử dụng	~200-300 MB RAM

CHƯƠNG 6: KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết luận

- Đã xây dựng thành công hệ thống website phân loại bệnh cây trồng qua ảnh lá cây sử dụng AI
- Model MobileNetV2 đạt **độ chính xác ~99%** trên bộ dữ liệu PlantVillage (15 lớp)
- Sử dụng **ONNX Runtime** để triển khai model nhẹ, chạy trên CPU mà không cần GPU
- Giao diện web hiện đại (dark mode, glassmorphism), hỗ trợ cả upload ảnh và chụp camera trực tiếp
- Hệ thống **responsive**, hoạt động tốt trên cả desktop và mobile

6.2. Hạn chế

- Chỉ hỗ trợ 3 loại cây trồng (cà chua, ớt chuông, khoai tây) với 15 lớp bệnh
- Dữ liệu huấn luyện (PlantVillage) chủ yếu là ảnh chụp trong điều kiện lý tưởng (nền trắng, đủ sáng), có thể giảm accuracy khi chụp thực tế ngoài đồng
- Lịch sử dự đoán lưu in-memory (mất khi restart server), chưa có database
- Chưa có chức năng xác thực người dùng (authentication)

6.3. Hướng phát triển

1. **Mở rộng dataset:** Thêm nhiều loại cây trồng và bệnh (lúa, ngô, cam, chanh...)
2. **Thu thập dữ liệu thực tế:** Ảnh chụp ngoài đồng ruộng để cải thiện khả năng tổng quát
3. **Tích hợp database:** PostgreSQL/MongoDB để lưu trữ lịch sử vĩnh viễn
4. **Authentication:** Đăng nhập/đăng ký để quản lý dữ liệu theo người dùng
5. **Ứng dụng mobile:** Phát triển app React Native/Flutter
6. **IoT Integration:** Kết nối camera giám sát vườn tự động phát hiện bệnh

7. **Model nâng cao:** Thử nghiệm Vision Transformer (ViT), YOLOv8 cho object detection (phát hiện vùng bệnh trên lá)
8. **Triển khai cloud:** Deploy trên AWS/GCP/Azure để phục vụ quy mô lớn

PHỤ LỤC

A. Cấu trúc thư mục dự án

```
tomato_plant_diseases/
├── backend/                                # FastAPI backend
│   ├── main.py                            # Server chính + API endpoints (147 dòng)
│   ├── predictor.py                       # Load model ONNX + inference logic (178 dòng)
│   ├── requirements.txt                   # Python dependencies (6 packages)
│   └── uploads/                           # Ảnh đã upload (auto-generated)
├── frontend/                              # Next.js 16 frontend
│   ├── src/app/
│   │   ├── globals.css                   # Design system (1012 dòng CSS)
│   │   ├── layout.tsx                    # Root layout + SEO metadata
│   │   ├── page.tsx                      # Trang Phân tích (463 dòng TSX)
│   │   └── dashboard/
│   │       └── page.tsx                  # Dashboard – Thống kê (323 dòng TSX)
│   ├── package.json                      # Node.js dependencies
│   ├── tsconfig.json                     # TypeScript config
│   └── next.config.ts                     # Next.js config
├── models/                               # AI model artifacts
│   ├── plant_disease_model.onnx          # ONNX model cho deployment (16.2 MB)
│   ├── best_model.pth                    # PyTorch weights (16.2 MB)
│   ├── efficientnet_model.pth             # EfficientNet weights (16.4 MB)
│   ├── class_names.json                  # 15 class names
│   ├── model_config.json                 # Model configuration
│   ├── learning_curves.png               # Training plots
│   └── __results__files/                  # Biểu đồ kết quả training (8 files)
├── training/                             # Jupyter notebooks huấn luyện
│   ├── plant-disease-classification-efficientnet-resnet50.ipynb
│   └── plant-disease-classifier-pytorch-custom-cnn.ipynb
├── data-testing/                          # Ảnh test mẫu (4 ảnh)
├── Baocao/                               # Báo cáo đồ án
│   ├── BAO_CA0_TTTN_PhanLoaiBenhLaCaChua.docx
│   ├── Huong-dan-trinh-bay-bao-cau-TTTN.docx
│   └── tao_bao_cao.py                    # Script tạo báo cáo tự động
├── YeuCau/                               # Đề cương đồ án
│   └── NguyenHuuKhanhTung_223630722_Decuong DATN-Cử nhân_V2.docx
└── README.md                             # README dự án (221 dòng)
```

B. Chi tiết 15 loại bệnh — mô tả & khuyến nghị

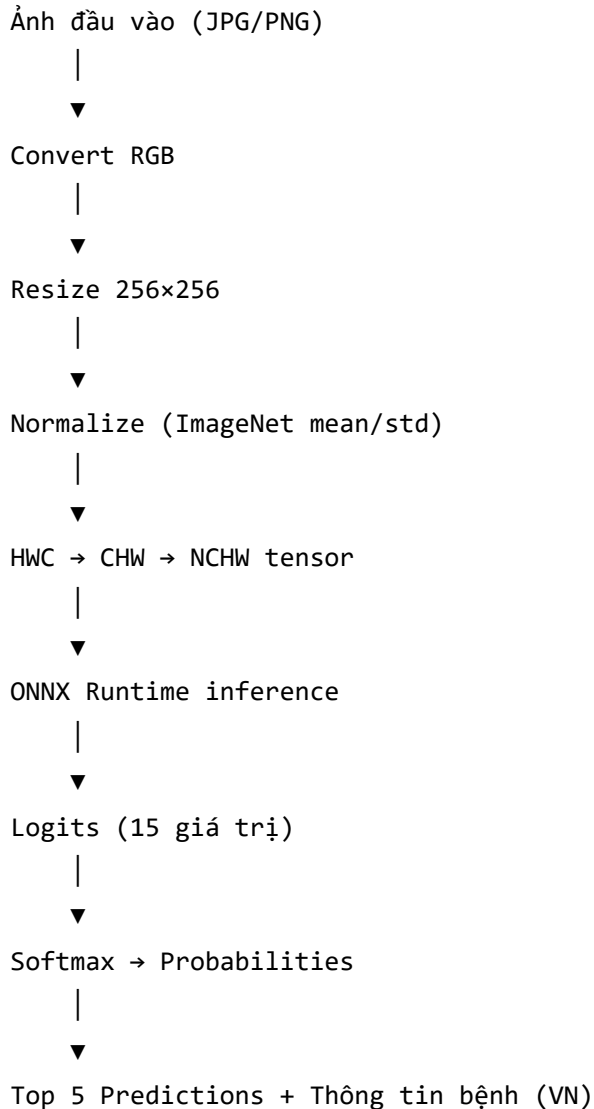
#	Bệnh	Tác nhân	Triệu chứng	Khuyến nghị xử lý
1	Ớt chuông — Đốm vi khuẩn	Vi khuẩn <i>Xanthomonas campestris</i>	Đốm nâu đen trên lá	Giống kháng bệnh, phun thuốc gốc đồng, luân canh
2	Ớt chuông — Khỏe mạnh	—	Không có triệu chứng	Chăm sóc bình thường
3	Khoai tây — Cháy lá sớm	Nấm <i>Alternaria solani</i>	Vòng đồng tâm trên lá	Phun thuốc trừ nấm, loại bỏ lá bệnh
4	Khoai tây — Cháy lá muộn	Nấm <i>Phytophthora infestans</i>	Vết bầm nước trên lá	Phun thuốc phòng ngừa, tránh tưới lên lá
5	Khoai tây — Khỏe mạnh	—	Không có triệu chứng	Chăm sóc bình thường
6	Cà chua — Đốm vi khuẩn	Vi khuẩn <i>Xanthomonas</i>	Đốm nước nhỏ trên lá	Phun thuốc gốc đồng, luân canh
7	Cà chua — Cháy lá sớm	Nấm <i>Alternaria solani</i>	Vòng đồng tâm nâu trên lá già	Cắt lá bệnh, phun thuốc trừ nấm
8	Cà chua — Cháy lá muộn	Nấm <i>Phytophthora infestans</i>	Lan nhanh trong ẩm ướt	Phun phòng ngừa, tránh tưới buổi tối
9	Cà chua — Mốc lá	Nấm <i>Passalora fulva</i>	Mốc vàng-xanh mặt dưới lá	Tăng thông thoáng, giảm độ ẩm
10	Cà chua — Đốm lá Septoria	Nấm <i>Septoria lycopersici</i>	Đốm tròn nhỏ, tâm xám	Cắt lá nhiễm, phun thuốc trừ nấm
11	Cà chua — Nhện đỏ hai chấm	Nhện đỏ (Spider mites)	Lá vàng, mạng nhện nhỏ	Phun nước mạnh rửa lá, thuốc trừ nhện
12	Cà chua — Đốm mục tiêu	Nấm <i>Corynespora cassiicola</i>	Đốm tròn đồng tâm	Phun thuốc trừ nấm, luân canh
13	Cà chua — Virus xoắn vàng	Virus TYLCV (bọ phấn trắng truyền)	Lá xoắn vàng, cây còi cọc	Diệt bọ phấn, lưới chắn côn trùng

#	Bệnh	Tác nhân	Triệu chứng	Khuyến nghị xử lý
	lá			
14	Cà chua — Virus khảm	Virus TMV	Vân khảm vàng- xanh, biến dạng lá	Loại bỏ cây bệnh, khử trùng dụng cụ
15	Cà chua — Khỏe mạnh	—	Không có triệu chứng	Chăm sóc bình thường, theo dõi định kỳ

C. Hướng dẫn cài đặt & chạy

- 1. **Yêu cầu:** Python ≥ 3.10, Node.js ≥ 18, npm ≥ 9
- 2. **Backend:** cd backend && pip install -r requirements.txt && python main.py
- 3. **Frontend:** cd frontend && npm install && npm run dev
- 4. **Truy cập:** http://localhost:3000 — Phân tích (/), Dashboard (/dashboard)

D. Pipeline xử lý ảnh (Inference)



Tip

Ghi chú cho người viết báo cáo:

- Các ô đánh dấu "(kết quả)" cần kiểm tra lại trong notebook training để điền chính xác
- Cần chụp **screenshot** các trang web (Trang Phân tích, Dashboard) để minh họa
- Nên thêm **Confusion Matrix** và **Classification Report** từ kết quả training
- File đề cương chi tiết nằm tại:
YeuCau/NguyenHuuKhanhTung_223630722_Decuong DATN-Cử nhân_V2.docx
- File hướng dẫn trình bày báo cáo nằm tại: Baocao/Huong-dan-trinh-bay-bao-cai-TTTN.docx